

Introduction to Docker

Session 41 · 07 August 2024

What is Docker?

Docker is a platform designed to help developers build, share, and run container applications.

Why Do We Need Docker?

Consistency Across Environments

Problem: Applications often behave differently in development, testing, and production environments due to variations in configurations, dependencies, and infrastructure.

Solution: Docker containers encapsulate all the necessary components, ensuring the application runs consistently across all environments.

Isolation

Problem: Running multiple applications on the same host can lead to conflicts, such as dependency clashes or resource contention.

Solution: Docker provides isolated environments for each application, preventing interference and ensuring stable performance.

Scalability

Problem: Scaling applications to handle increased load can be challenging, requiring manual intervention and configuration.

Solution: Docker makes it easy to scale applications horizontally by running multiple container instances, allowing for quick and efficient scaling.

Docker Engine

Docker Engine is the core component of the Docker platform, responsible for creating, running, and managing Docker containers. It serves as the runtime that powers Docker's containerization capabilities.

Components of Docker Engine

1. Docker Daemon (dockerd)

The Docker daemon is the background service running on the host machine. It manages Docker objects such as images, containers, networks, and volumes. It listens for Docker API requests and processes them, handling container lifecycle operations (start, stop, restart, etc.).

2. Docker CLI (docker)

The Docker Command Line Interface (CLI) is the tool that users interact with to communicate with the Docker daemon. Users run Docker commands through the CLI to perform tasks like building images, running containers, and managing Docker resources.

3. REST API

The Docker REST API allows communication between the Docker CLI and the Docker daemon. It also enables programmatic interaction with Docker. Developers can use the API to automate Docker operations or integrate Docker functionality into their applications.

Docker Image

A Docker image is a lightweight, stand-alone, and executable software package that includes everything needed to run a piece of software, such as the code, runtime, libraries, environment variables, and configuration files. Images are used to create Docker containers, which are instances of these images.

Components of a Docker Image

Base Image: The starting point for building an image. It could be a minimal OS image like alpine, a full-fledged OS like ubuntu, or even another application image like python or node.

Application Code: The actual code and files necessary for the application to run.

Dependencies: Libraries, frameworks, and packages required by the application.

Metadata: Information about the image, such as environment variables, labels, and exposed ports.

Docker Image Lifecycle

Creation: Images are created using the docker build command, which processes the instructions in a Dockerfile to create the image layers.

Storage: Images are stored locally on the host machine. They can also be pushed to and pulled from Docker registries like Docker Hub, AWS ECR, or Google Container Registry.

Distribution: Images can be shared by pushing them to a Docker registry, allowing others to pull and use the same image.

Execution: Images are executed by running containers, which are instances of these images.

Dockerfile

A Dockerfile is a text file that contains a series of instructions used to build a Docker image. Each instruction in a Dockerfile creates a layer in the image, allowing for efficient image creation and reuse of layers. Dockerfiles are used to automate the image creation process, ensuring consistency and reproducibility.

Key Components of a Dockerfile

1. FROM — Specifies the starting point for the image. Example: FROM ubuntu:20.04

- 2. LABEL** — Adds metadata to the image. Example: LABEL version="1.0" description="My application"
- 3. RUN** — Executes commands during the build process. Example: RUN apt-get update && apt-get install -y python3
- 4. COPY** — Copies files from the host to the image. Example: COPY . /app
- 5. ENV** — Sets environment variables. Example: ENV PATH /app/bin:\$PATH
- 6. WORKDIR** — Sets the working directory. Example: WORKDIR /app
- 7. EXPOSE** — Exposes network ports. Example: EXPOSE 8080
- 8. CMD** — Provides a default command when the container starts. Example: CMD ["python", "app.py"]
- 9. VOLUME** — Creates a mount point. Example: VOLUME ["/data"]
- 10. ARG** — Defines build-time variables. Example: ARG VERSION=1.0

Example Dockerfile

```
FROM python:3.8-slim

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 80

ENV NAME World

CMD ["python", "app.py"]
```

Docker Container

A Docker container is a lightweight, portable, and isolated environment that encapsulates an application and its dependencies, allowing it to run consistently across different computing environments. Containers are created from Docker images, which are immutable and contain all the necessary components for the application to run.

Registry

A Docker registry is a service that stores and distributes Docker images. It acts as a repository where users can push, pull, and manage Docker images. Docker Hub is the most well-known public registry, but private registries can also be set up to securely store and manage images within an organization.

Key Components of a Docker Registry

Repositories: A repository is a collection of related Docker images, typically different versions of the same application. Each repository can hold multiple tags representing different versions.

Tags: Tags are used to version images within a repository. For example, myapp:1.0, myapp:2.0, and myapp:latest are tags for different versions.

Types of Docker Registries

Docker Hub: The default public registry provided by Docker, which hosts a vast number of public images and also supports private repositories. URL: hub.docker.com. Use case: publicly sharing images and accessing pre-built images from the community.

Private Registries: Custom registries set up by organizations to securely store and manage their own Docker images. Use case: ensuring security and control over image distribution within an organization.

Third-Party Registries: Examples include Amazon Elastic Container Registry (ECR), Google Container Registry (GCR), and Azure Container Registry (ACR). Use case: integrating with cloud platforms for seamless deployment and management.

Benefits of Using Docker Registries

- **Centralized Image Management:** Provides a centralized location to store and manage Docker images, making it easier to organize and distribute them.
 - **Version Control:** Using tags, registries allow version control of images, enabling easy rollback to previous versions.
 - **Collaboration:** Public registries like Docker Hub facilitate collaboration by allowing users to share images with the community or within teams.
 - **Security:** Private registries ensure that sensitive images are stored securely and access is controlled within an organization.
 - **Integration with CI/CD:** Registries integrate seamlessly with CI/CD pipelines, automating the process of building, storing, and deploying Docker images.
-

Use Cases

Microservices Architecture

Microservices break down applications into smaller, independent services, each running in its own container. This simplifies deployment, scaling, and maintenance — each service can be developed, updated, and deployed independently.

Continuous Integration and Deployment (CI/CD)

Docker ensures a consistent environment from development through testing to production, streamlining the CI/CD pipeline, reducing environment discrepancies, and speeding up testing and deployment.

Cloud Migration

Containerizing applications simplifies moving them to the cloud, allows applications to run consistently across different cloud providers, and optimizes resource usage.

Scalable Web Applications

Deploying web applications in containers makes scaling up or down based on traffic simple, ensures consistent deployment, and enhances resource utilization.

Testing and QA

Docker creates consistent environments for testing, ensuring tests run in environments identical to production and facilitating automated testing.

Machine Learning and AI

Deploying ML models in containers ensures runtime consistency, simplifies scaling of model training and inference, and facilitates collaboration and reproducibility.

API Development and Deployment

Developing and deploying APIs in containers ensures they run consistently across environments, simplifies scaling, and improves deployment speed and reliability.